

BCA
III
SEMESTER

BCA
III SEMESTER



ACCORDING TO
**LATEST
SYLLABUS**

JAI NARAYAN VYAS
UNIVERSITY, JODHPUR

JAVA PROGRAMMING

A COMPLETE TEXTBOOK FOR BCA STUDENTS

COVERS

-  Java Fundamentals
-  Object-Oriented Programming
-  Packages, Interfaces & Inheritance
-  Exception Handling & Multithreading
-  File Handling & Collections Framework
-  Java I/O, Applets & GUI Programming
-  JDBC & Database Connectivity
-  Practical Programs & Case Studies



EASY
EXPLANATION



CONCEPTS WITH
EXAMPLES



EXAM
FOCUSED



DIAGRAMS & TABLES
FOR BETTER
UNDERSTANDING

• LEARN TODAY, LEAD TOMORROW •

UNIT - I: OOP के सिद्धांत और Java का परिचय

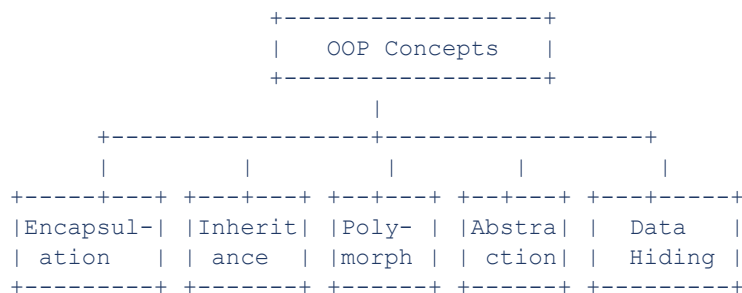
1.1 OOP (Object Oriented Programming) क्या है?

Object Oriented Programming (OOP) एक programming paradigm है जो real-world objects के concept पर आधारित है। इसमें program को objects और classes के रूप में organize किया जाता है।

OOP के मुख्य सिद्धांत (Principles of OOP):

सिद्धांत	अर्थ	उदाहरण
Encapsulation (एनकैप्सुलेशन)	Data और Methods को एक unit में बंद करना	Bank Account class
Inheritance (इनहेरिटेन्स)	एक class का गुण दूसरी class को मिलना	Animal → Dog
Polymorphism (पॉलिमॉर्फिज्म)	एक ही नाम के अलग-अलग रूप	draw() - circle, square
Abstraction (एब्स्ट्रैक्शन)	जरूरी विवरण दिखाना, अनावश्यक छुपाना	Car - only steering visible
Data Hiding (डेटा हाइडिंग)	Data को बाहर से छुपाना	private variables
Overloading (ओवरलोडिंग)	एक ही नाम के अनेक methods	add(int), add(double)

OOP का आरेख (Diagram):



1.2 Encapsulation (एनकैप्सुलेशन)

Encapsulation का अर्थ है data (variables) और methods को एक ही class में बंद करना और data को private रखकर उसे केवल methods के द्वारा access करना।

Program: Encapsulation का उदाहरण

```
class BankAccount {
    private double balance; // private - बाहर से access नहीं
    private String owner;

    // Constructor
    BankAccount(String name, double amount) {
        owner = name;
    }
}
```

```

        balance = amount;
    }

    // Getter method - balance देखने के लिए
    public double getBalance() {
        return balance;
    }

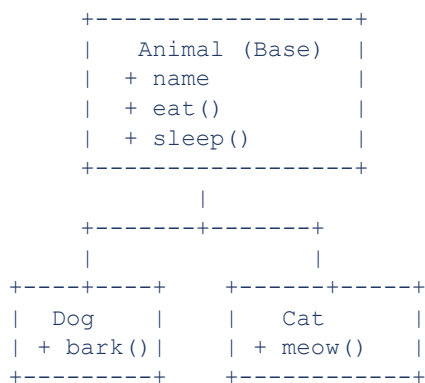
    // Method - पैसे जमा करने के लिए
    public void deposit(double amount) {
        if(amount > 0)
            balance += amount;
    }
}

class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Rahul", 5000);
        acc.deposit(2000);
        System.out.println("Balance: " + acc.getBalance());
        // Output: Balance: 7000.0
    }
}

```

1.3 Inheritance (इनहेरिटेंस)

Inheritance वह प्रक्रिया है जिसमें एक class (child/derived class) दूसरी class (parent/base class) के गुण और methods को inherit (प्राप्त) करती है।



```

class Animal {
    String name;
    void eat() { System.out.println(name + " खाना खा रहा है"); }
}

class Dog extends Animal { // extends = inheritance
    void bark() { System.out.println(name + " भौंक रहा है"); }
}

class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.name = "Tommy";
        d.eat(); // Output: Tommy खाना खा रहा है
        d.bark(); // Output: Tommy भौंक रहा है
    }
}

```

```
}
```

1.4 Polymorphism (पॉलिमॉर्फिज्म)

Polymorphism का अर्थ है "अनेक रूप"। इसमें एक ही method अलग-अलग classes में अलग-अलग तरीके से काम करता है।

```
class Shape {
    void draw() { System.out.println("आकार बनाएं"); }
}
class Circle extends Shape {
    void draw() { System.out.println("वृत्त बनाएं"); }
}
class Rectangle extends Shape {
    void draw() { System.out.println("आयत बनाएं"); }
}
class Main {
    public static void main(String[] args) {
        Shape s;
        s = new Circle();    s.draw();    // वृत्त बनाएं
        s = new Rectangle(); s.draw();    // आयत बनाएं
    }
}
```

1.5 JAVA Program Structure (Java प्रोग्राम की संरचना)

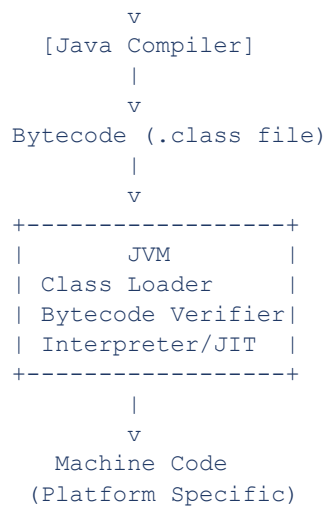
```
+-----+
|  Java Program Structure  |
|                           |
|  1. Package Statement   (Optional) |
|  2. Import Statements  (Optional) |
|  3. Class Declaration   |
|  +-----+ |
|  | 4. Variables (Data Members) | | |
|  | 5. Constructor              | |
|  | 6. Methods                  | |
|  | +-----+ | |
|  | | main() method - Entry Point | |
|  | +-----+ | |
|  +-----+ |
+-----+
```

```
// पहला Java Program
package mypackage;           // Package (वैकल्पिक)
import java.util.Scanner;    // Import (वैकल्पिक)

public class HelloWorld {    // Class Declaration
    // main method - program यहाँ से शुरू होता है
    public static void main(String[] args) {
        System.out.println("नमस्ते Java!");
    }
}
// Output: नमस्ते Java!
```

1.6 Java Virtual Machine (JVM)

Java Source Code (.java)
|



JVM Java bytecode को machine code में convert करता है। यही कारण है कि Java 'Write Once, Run Anywhere' है।

1.7 Data Types (डेटा प्रकार)

Data Type	Size	Range	उदाहरण
byte	1 byte	-128 to 127	byte b = 10;
short	2 bytes	-32768 to 32767	short s = 1000;
int	4 bytes	-2 ³¹ to 2 ³¹ -1	int n = 50000;
long	8 bytes	-2 ⁶³ to 2 ⁶³ -1	long l = 99999L;
float	4 bytes	~3.4e38	float f = 3.14f;
double	8 bytes	~1.7e308	double d = 3.14;
char	2 bytes	0 to 65535	char c = 'A';
boolean	1 bit	true/false	boolean b = true;

1.8 Operators (ऑपरेटर्स)

Arithmetic Operators (अंकगणित ऑपरेटर):

```

int a = 10, b = 3;
System.out.println(a + b); // 13 (जोड़)
System.out.println(a - b); // 7 (घटाव)
System.out.println(a * b); // 30 (गुणा)
System.out.println(a / b); // 3 (भाग)
System.out.println(a % b); // 1 (शेष)

```

Relational Operators (तुलनात्मक ऑपरेटर):

```

int x = 5, y = 8;
System.out.println(x > y); // false
System.out.println(x < y); // true
System.out.println(x == y); // false
System.out.println(x != y); // true
System.out.println(x >= y); // false
System.out.println(x <= y); // true

```

Logical Operators (तार्किक ऑपरेटर):

```
boolean p = true, q = false;
System.out.println(p && q); // false (AND - दोनों सच हों)
System.out.println(p || q); // true (OR - कोई एक सच हो)
System.out.println(!p); // false (NOT - उल्टा)
```

Increment/Decrement Operators:

```
int n = 5;
System.out.println(n++); // 5 (पहले print, फिर बढ़ाएं)
System.out.println(n); // 6
System.out.println(++n); // 7 (पहले बढ़ाएं, फिर print)
System.out.println(n--); // 7 (पहले print, फिर घटाएं)
```

□ याद रखें: `i++` Post-increment है (पहले use, फिर बढ़ाओ), `++i` Pre-increment है (पहले बढ़ाओ, फिर use)

1.9 Type Casting (टाइप कास्टिंग)

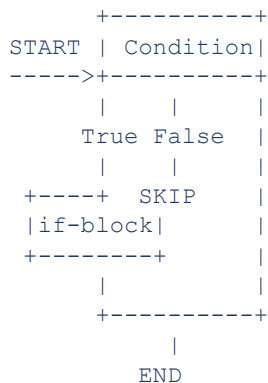
```
// Implicit (Widening) Casting - छोटे से बड़े में
int i = 100;
long l = i; // automatic
double d = l; // automatic

// Explicit (Narrowing) Casting - बड़े से छोटे में
double pi = 3.14;
int n = (int) pi; // explicit cast - decimal part खो जाएगा
System.out.println(n); // Output: 3
```

UNIT - II: Control Statements (नियंत्रण कथन)

2.1 if Statement

if statement किसी condition के सत्य होने पर code execute करती है।

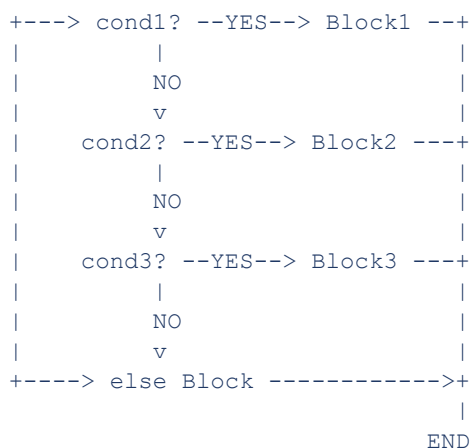


```
int marks = 75;
if (marks >= 60) {
    System.out.println("प्रथम श्रेणी");
}
// Output: प्रथम श्रेणी
```

2.2 if...else Statement

```
int age = 17;
if (age >= 18) {
    System.out.println("मतदान कर सकते हैं");
} else {
    System.out.println("मतदान नहीं कर सकते");
}
// Output: मतदान नहीं कर सकते
```

2.3 else...if Ladder (सीढ़ी)



```
int marks = 72;
if (marks >= 90)      System.out.println("A Grade");
else if (marks >= 75) System.out.println("B Grade");
else if (marks >= 60) System.out.println("C Grade");
else if (marks >= 50) System.out.println("D Grade");
else                  System.out.println("Fail");
```



```
// Output: C Grade
```

2.4 Nested if...else

```
int a = 10, b = 20, c = 15;
if (a > b) {
    if (a > c) System.out.println("a सबसे बड़ा है");
    else      System.out.println("c सबसे बड़ा है");
} else {
    if (b > c) System.out.println("b सबसे बड़ा है");
    else      System.out.println("c सबसे बड़ा है");
}
// Output: b सबसे बड़ा है
```

2.5 Switch Statement

```
switch(expression)
|
+---+---+---+---+
|   |   |   |   |
case1 case2 case3 default
|   |   |   |
block block block block
|   |   |   |
break break break
|
END

int day = 3;
switch(day) {
    case 1: System.out.println("सोमवार"); break;
    case 2: System.out.println("मंगलवार"); break;
    case 3: System.out.println("बुधवार"); break;
    case 4: System.out.println("गुरुवार"); break;
    case 5: System.out.println("शुक्रवार"); break;
    default: System.out.println("सप्ताहांत");
}
// Output: बुधवार
```

□ break न लगाने पर Fall-through होता है - अगले case भी execute होते हैं

2.6 Ternary Operator (? :)

```
// Syntax: condition ? value_if_true : value_if_false
int a = 10, b = 20;
int max = (a > b) ? a : b;
System.out.println("बड़ा: " + max);
// Output: बड़ा: 20
```

2.7 Loops - While Loop

```
START
|
v
[Condition] --FALSE--> END
```



```

|
TRUE
|
v
[Loop Body]
|
v
[Update]
|
+---> back to Condition

```

```

// 1 से 5 तक print करें
int i = 1;
while (i <= 5) {
    System.out.print(i + " ");
    i++;
}
// Output: 1 2 3 4 5

```

2.8 do...while Loop

```

// कम से कम एक बार execute होता है
int i = 1;
do {
    System.out.print(i + " ");
    i++;
} while (i <= 5);
// Output: 1 2 3 4 5

```

2.9 for Loop

```

// Syntax: for(initialization; condition; update)
for (int i = 1; i <= 10; i++) {
    System.out.print(i + " ");
}
// Output: 1 2 3 4 5 6 7 8 9 10

// Multiplication Table (पहाड़ा)
int n = 5;
for (int i = 1; i <= 10; i++) {
    System.out.println(n + " x " + i + " = " + (n*i));
}

```

Loop	कब Use करें	Condition Check
while	जब पहले से न पता हो कितनी बार	शुरू में
do-while	कम से कम एक बार जरूर चलाना हो	अंत में
for	जब पता हो कितनी बार चलाना है	शुरू में

2.10 Jumps in Loops

```

// break - loop तुरंत बंद
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;
    System.out.print(i + " ");
}

```

```
}  
// Output: 1 2 3 4  
  
// continue - current iteration skip  
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue; // even skip  
    System.out.print(i + " ");  
}  
// Output: 1 3 5 7 9
```

2.11 Labelled Loops

```
outer:                // label  
for (int i = 1; i <= 3; i++) {  
    for (int j = 1; j <= 3; j++) {  
        if (j == 2) continue outer; // बाहरी loop को continue  
        System.out.println(i + "," + j);  
    }  
}  
// Output:  
// 1,1  
// 2,1  
// 3,1
```

UNIT - III: Classes and Objects (क्लास और ऑब्जेक्ट)

3.1 Class Define करना

```
+-----+
|          CLASS          |
+-----+
| Data Members (Variables/Fields) |
| - int rollNo;              |
| - String name;            |
| - double marks;          |
+-----+
| Member Functions (Methods) |
| - Student() // Constructor |
| - void display()          |
| - double getAverage()     |
+-----+
```

```
class Student {
    // Data Members
    int rollNo;
    String name;
    double marks;

    // Constructor
    Student(int r, String n, double m) {
        rollNo = r;
        name = n;
        marks = m;
    }

    // Method
    void display() {
        System.out.println("Roll: " + rollNo + ", Name: " + name + ", Marks: " + marks);
    }
}
```

3.2 Object बनाना (Creating Objects)

```
class Main {
    public static void main(String[] args) {
        // Object बनाना
        Student s1 = new Student(101, "Priya", 85.5);
        Student s2 = new Student(102, "Rahul", 78.0);

        // Method call करना
        s1.display(); // Roll: 101, Name: Priya, Marks: 85.5
        s2.display(); // Roll: 102, Name: Rahul, Marks: 78.0
    }
}
```

3.3 Constructors (कंस्ट्रक्टर)

Constructor एक special method है जो object बनाते समय automatically call होता है। इसका नाम class के नाम जैसा होता है।

```
class Box {
```

```

    int length, width, height;

    // Default Constructor (कोई argument नहीं)
    Box() {
        length = width = height = 0;
    }

    // Parameterized Constructor (arguments के साथ)
    Box(int l, int w, int h) {
        length = l; width = w; height = h;
    }

    int volume() { return length * width * height; }
}

class Main {
    public static void main(String[] args) {
        Box b1 = new Box();           // Default
        Box b2 = new Box(3, 4, 5);    // Parameterized
        System.out.println(b2.volume()); // 60
    }
}

```

3.4 Method Overloading (मेथड ओवरलोडिंग)

एक ही class में same नाम के methods जिनके parameters अलग हों।

```

class Calculator {
    // Same नाम, अलग parameters
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }

    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(5, 3));           // 8
        System.out.println(c.add(2.5, 3.7));       // 6.2
        System.out.println(c.add(1, 2, 3));        // 6
    }
}

```

3.5 Static Members (स्टैटिक सदस्य)

```

class Counter {
    static int count = 0; // सभी objects share करते हैं
    int id;

    Counter() {
        count++; // हर object बनने पर count बढ़ता है
        id = count;
    }

    static void showCount() { // Static method
        System.out.println("Total objects: " + count);
    }
}

```

```

class Main {
    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();
        Counter c3 = new Counter();
        Counter.showCount(); // Total objects: 3
    }
}

```

3.6 Access Specifiers (एक्सेस स्पेसिफायर्स)

Access Modifier	Same Class	Same Package	Subclass	Other
public	✓हाँ	✓हाँ	✓हाँ	✓हाँ
protected	✓हाँ	✓हाँ	✓हाँ	✗ नहीं
default (none)	✓हाँ	✓हाँ	✗ नहीं	✗ नहीं
private	✓हाँ	✗ नहीं	✗ नहीं	✗ नहीं

3.7 Inheritance - Derived Classes

```

class Vehicle { // Base Class
    String brand;
    int speed;

    void start() { System.out.println(brand + " चालू हो गई"); }
    void stop() { System.out.println(brand + " बंद हो गई"); }
}

class Car extends Vehicle { // Derived Class
    int doors;

    void honk() { System.out.println("बीप बीप!"); }
}

class Main {
    public static void main(String[] args) {
        Car c = new Car();
        c.brand = "Maruti"; c.speed = 120; c.doors = 4;
        c.start(); // Maruti चालू हो गई
        c.honk(); // बीप बीप!
        c.stop(); // Maruti बंद हो गई
    }
}

```

3.8 Method Overriding (मैथड ओवरराइडिंग)

```

class Animal {
    void sound() { System.out.println("जानवर आवाज करता है"); }
}

```

```

class Dog extends Animal {
    @Override
    void sound() { System.out.println("कुत्ता: भौं भौं"); } // Override
}

class Cat extends Animal {
    @Override
    void sound() { System.out.println("बिल्ली: म्याऊं"); } // Override
}

class Main {
    public static void main(String[] args) {
        Animal a;
        a = new Dog(); a.sound(); // कुत्ता: भौं भौं
        a = new Cat(); a.sound(); // बिल्ली: म्याऊं
    }
}

```

3.9 Abstract Classes (एब्स्ट्रेक्ट क्लास)

Abstract class वह class है जिसे directly instantiate नहीं किया जा सकता। इसमें abstract methods हो सकते हैं जिन्हें subclass में implement करना जरूरी है।

```

abstract class Shape {
    abstract double area(); // Abstract method - body नहीं
    void display() { System.out.println("क्षेत्रफल: " + area()); }
}

class Circle extends Shape {
    double radius;
    Circle(double r) { radius = r; }

    @Override
    double area() { return 3.14 * radius * radius; }
}

class Main {
    public static void main(String[] args) {
        Shape s = new Circle(5);
        s.display(); // क्षेत्रफल: 78.5
    }
}

```

3.10 Final Keyword

```

// 1. final variable - constant
final double PI = 3.14159;
// PI = 3.14; // Error! बदल नहीं सकते

// 2. final method - override नहीं हो सकता
class Parent {
    final void show() { System.out.println("यह बदल नहीं सकता"); }
}

// 3. final class - extend नहीं हो सकती

```

```
final class ImmutableClass {  
    // यह class inherit नहीं हो सकती  
}
```


UNIT - IV: Arrays, Strings and Interfaces

4.1 Arrays (सरणी)

Array एक ही data type के elements का collection है जो continuous memory locations में store होते हैं।

```
int[] arr = {10, 20, 30, 40, 50};
```

```
Index:  [0]  [1]  [2]  [3]  [4]
        +-----+-----+-----+-----+
Value:  | 10 | 20 | 30 | 40 | 50 |
        +-----+-----+-----+-----+
```

```
Memory: 1000  1004  1008  1012  1016
```

```
// One Dimensional Array
int[] marks = new int[5]; // Declaration
marks[0] = 85;
marks[1] = 92;
marks[2] = 78;

// Short form
int[] scores = {90, 85, 78, 92, 88};

// Array traverse करना
for (int i = 0; i < scores.length; i++) {
    System.out.print(scores[i] + " ");
}
// Output: 90 85 78 92 88

// Enhanced for loop
for (int s : scores) {
    System.out.print(s + " ");
}
```

4.2 Two Dimensional Arrays (द्विआयामी सरणी)

```
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};
```

```
          Col 0  Col 1  Col 2
Row 0 |  1  |  2  |  3  |
Row 1 |  4  |  5  |  6  |
Row 2 |  7  |  8  |  9  |
```

```
int[][] matrix = {{1,2,3},{4,5,6},{7,8,9}};

// Matrix print करना
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + "\t");
    }
    System.out.println();
}

// Matrix Addition Program
int[][] a = {{1,2},{3,4}};
int[][] b = {{5,6},{7,8}};
int[][] sum = new int[2][2];
for(int i=0; i<2; i++)
    for(int j=0; j<2; j++)
        sum[i][j] = a[i][j] + b[i][j];
```

4.3 Strings (स्ट्रिंग्स)

Java में String एक class है (primitive type नहीं)। String objects immutable होते हैं।

```
String s1 = "नमस्ते";
String s2 = new String("Java");

// String Methods
String name = "Hello World";
System.out.println(name.length());           // 11
System.out.println(name.toUpperCase());       // HELLO WORLD
System.out.println(name.toLowerCase());       // hello world
System.out.println(name.substring(6));        // World
System.out.println(name.charAt(0));           // H
System.out.println(name.indexOf('o'));        // 4
System.out.println(name.contains("World"));   // true
System.out.println(name.replace('l', 'L'));   // HeLLo WorLd
System.out.println(name.trim());              // spaces हटाना
```

String Method	काम	उदाहरण
length()	लंबाई बताता है	"Java".length() = 4
charAt(i)	i-वें character को return करता है	"Java".charAt(1) = 'a'
substring(a,b)	a से b तक string	"Hello".substring(1,3) = "el"
equals()	दो strings compare करता है	s1.equals(s2)
compareTo()	lexicographic comparison	s1.compareTo(s2)
toUpperCase()	सभी capital letters	"hi".toUpperCase() = "HI"
split()	string को parts में बाँटना	"a,b,c".split(",")

4.4 Interface (इंटरफ़ेस)

Interface एक blueprint है जो only abstract methods और constants define करता है। यह multiple inheritance को support करता है।

```
+-----+ +-----+
| <<interface>> | | <<interface>> |
| Printable      | | Saveable      |
| + print()      | | + save()       |
+-----+ +-----+

      |           |
      +-----+
      |
      +-----+
      | Document  |
      | + print() impl |
      | + save()  impl |
      +-----+
```

```
// Interface define करना
interface Drawable {
    void draw(); // abstract by default
    int MAX_SIZE = 100; // constant
}
```

```
// Interface extend करना
interface Colorable extends Drawable {
    void fillColor(String color);
}

// Interface implement करना
class Circle implements Colorable {
    public void draw() {
        System.out.println("वृत्त बनाया");
    }
    public void fillColor(String color) {
        System.out.println(color + " रंग भरा");
    }
}

class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
        c.draw();           // वृत्त बनाया
        c.fillColor("लाल"); // लाल रंग भरा
    }
}
```

विशेषता	Abstract Class	Interface
Methods	Abstract + Concrete दोनों	केवल Abstract (Java 7 तक)
Variables	Any type	केवल public static final
Constructor	हो सकता है	नहीं हो सकता
Inheritance	Single	Multiple
Keyword	abstract class	interface

5.1 Packages (पैकेज)

```
java (root package)
|
+-- java.lang      (String, Math, System...)
+-- java.util      (Scanner, ArrayList...)
+-- java.io        (File, InputStream...)
+-- java.net       (Socket, URL...)
+-- java.awt       (Graphics, Button...)
+-- javax.swing    (JFrame, JButton...)

// Package बनाना
package com.myapp.utils; // पहली line होनी चाहिए

public class MathHelper {
    public static int square(int n) { return n * n; }
}

// Package use करना
import com.myapp.utils.MathHelper;

class Main {
    public static void main(String[] args) {
        System.out.println(MathHelper.square(5)); // 25
    }
}
```

5.2 Java System Packages

Package	उपयोग	मुख्य Classes
java.lang	Auto-import होता है	String, Math, System, Object
java.util	Utility classes	Scanner, ArrayList, HashMap, Date
java.io	Input/Output	File, BufferedReader, PrintWriter
java.net	Network programming	Socket, URL, HttpURLConnection
java.awt	GUI (Abstract Window)	Frame, Button, Label, TextField
javax.swing	Advanced GUI	JFrame, JButton, JTextField, JLabel

5.3 Exceptions (अपवाद/Exception Handling)

```

+----> try block ----> Exception आई? ---NO--> Normal Flow
|
|                               YES

```

```
class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int[] arr = new int[5];
            arr[10] = 50; // Error: index 10 exist नहीं करता
            System.out.println("यह print नहीं होगा");
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: " + e.getMessage());
        }
        catch (Exception e) {
            System.out.println("General Error: " + e);
        }
        finally {
            System.out.println("यह हमेशा print होगा");
        }
    }
}
```

Exception	कारण	उदाहरण
NullPointerException	null object पर operation	String s = null; s.length();
ArrayIndexOutOfBoundsException	Invalid array index	arr[10] जब size 5 हो
ArithmeticException	Mathematical error	5 / 0
NumberFormatException	String को number में convert error	Integer.parseInt("abc")
ClassCastException	Invalid type casting	(String) intObj
StackOverflowError	Infinite recursion	Recursive method without base

5.4 Custom Exception (अपना Exception बनाना)

```
// Custom Exception Class
class AgeException extends Exception {
    AgeException(String message) {
        super(message);
    }
}

class Voter {
    static void checkAge(int age) throws AgeException {
        if (age < 18)
            throw new AgeException("आयु 18 से कम है, मत नहीं दे सकते");
        System.out.println("मतदाता है!");
    }

    public static void main(String[] args) {
```

```

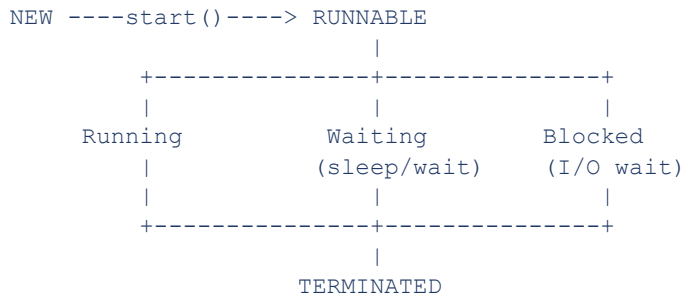
        try {
            checkAge(15);
        } catch (AgeException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

5.5 Threads (थ्रेड)

Thread program execution का smallest unit है। Java में एक साथ कई threads (multithreading) चल सकते हैं।

Thread Life Cycle:



```

// Method 1: Thread class extend करना
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread: " + i);
            try { Thread.sleep(500); } catch(Exception e) {}
        }
    }
}

// Method 2: Runnable Interface implement करना
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable: " + i);
        }
    }
}

class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        Thread t2 = new Thread(new MyRunnable());
        t1.start(); // Thread शुरू
        t2.start(); // दोनों साथ चलेंगे
    }
}

```

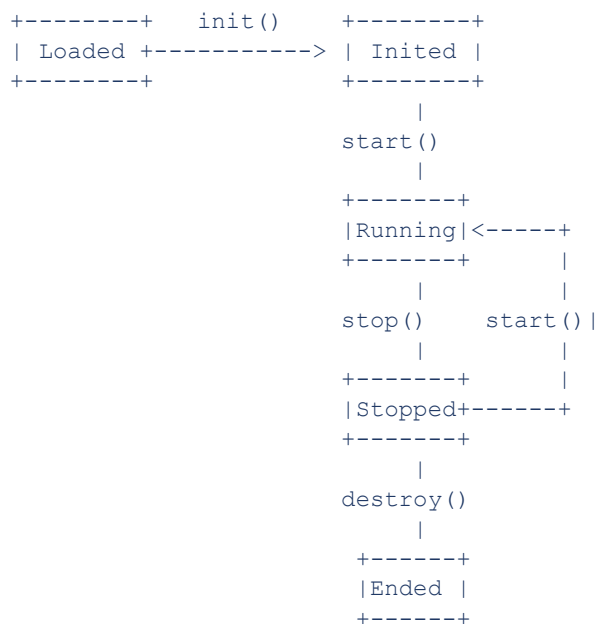
Thread Method	काम
start()	Thread शुरू करता है
run()	Thread का काम यहाँ लिखते हैं
sleep(ms)	Thread को ms मिलीसेकंड के लिए रोकता है

join()	एक thread दूसरे के पूरा होने का इंतजार करता है
getPriority()	Thread की priority देखता है
setName()	Thread को नाम देता है

5.6 Applets (एप्लेट)

Applet एक छोटा Java program है जो web browser में run होता है। यह java.applet.Applet class से extend होता है।

Applet Life Cycle:



```

import java.applet.*;
import java.awt.*;

public class HelloApplet extends Applet {

    // init() - एक बार शुरू में call होता है
    public void init() {
        setBackground(Color.yellow);
    }

    // start() - हर बार visible होने पर
    public void start() {
        System.out.println("Applet started");
    }

    // paint() - screen पर draw करने के लिए
    public void paint(Graphics g) {
        g.setColor(Color.red);
        g.setFont(new Font("Arial", Font.BOLD, 24));
        g.drawString("नमस्ते! Java Applet", 50, 100);
        g.drawOval(100, 150, 200, 100);
    }

    // stop() - hidden होने पर
    public void stop() { }
}

```

```
// destroy() - browser बंद होने पर  
public void destroy() { }  
  
}  
  
/* HTML में use करें:  
<applet code="HelloApplet.class" width=400 height=300>  
</applet>  
*/
```